HƯỚNG DẪN THAO TÁC CI/CD & ĐÓNG GÓI DOCKER

Dự Án Nghiên Cứu & Dự Báo Nhiệt Độ Ngắn Hạn Đa Trạm Tại Hà Nội (DINOv3 Forecaster)

Thông số kỹ thuật	Giá trị xác minh thực nghiệm	Ghi chú
Hệ điều hành	Windows 11 Pro 64-bit / Ubuntu 22.04 LTS	Đa nền tảng
Phiên bản Python	Python 3.12.3 (khóa trong requirements.lock)	Phụ thuộc cố định
Công cụ CI/CD	GitHub Actions (Runner ubuntu-latest)	Permissions: contents read
Phiên bản Docker	Docker Engine 26.x / Buildx v3.0	Chạy Non-Root (appuser)
Docker Hub Repository	nguyenthuanhoa/hanoi-temperature	Kho lưu trữ Public
Release Tag / Digest	v1.0.0 (sha256:8f4c1e0b...)	Chuẩn Semantic Versioning
Kết quả kiểm thử	49/49 Tests PASSED (100%)	Kiểm định toàn diện

1. Yêu Cầu Chuẩn Bị & Cấu Trúc Repository

Hệ thống yêu cầu tài khoản GitHub (có quyền quản trị repo) và tài khoản Docker Hub (ví dụ hanguynthun) để tự động hóa kiểm định và phát hành image.

Các file cốt lõi phục vụ CI/CD & Docker trong dự án:
• .github/workflows/ci.yml: Pipeline CI chạy linting, 49 unit tests và build dry-run.
• .github/workflows/release.yml: Pipeline CD kích hoạt khi tạo Git tag v*.
• Dockerfile: Biên dịch image trên nền Python 3.12 slim, chạy dưới quyền non-root appuser.
• .dockerignore: Ngăn chặn rò rỉ credential, private dataset, file cấu hình nhạy cảm.
• docs/docker_run.md & docs/release.md: Sổ tay hướng dẫn chi tiết và chiến lược rollback.

2. Cấu Hình Bảo Mật: Docker Hub Token & GitHub Secrets

Bước 2.1: Tạo Access Token trên Docker Hub:
1. Đăng nhập hub.docker.com → vào Account Settings → Security → New Access Token.
2. Đặt mô tả: github-actions-hanoi-weather với quyền hạn Read & Write.
3. Nhấn Generate và sao chép chuỗi token được cấp (Personal Access Token).

Bước 2.2: Cấu hình GitHub Secrets trong Repository:
1. Vào GitHub Repository → Settings → Secrets and variables → Actions.
2. Nhấn New repository secret để khai báo 2 biến bí mật:
   • DOCKERHUB_USERNAME: Tên tài khoản Docker Hub (ví dụ: hanguynthun).
   • DOCKERHUB_TOKEN: Chuỗi Access Token vừa tạo ở Bước 2.1.
Lưu ý: GitHub mã hóa các giá trị này bằng chuẩn AES-256 và tự động ẩn (masking) trong toàn bộ log.

Nguyên tắc an toàn: Tuyệt đối không lưu token vào mã nguồn, file commit, hoặc tài liệu công khai.

3. Quy Trình CI Tự Động: Trạng Thái Đỏ → Trạng Thái Xanh

Pipeline CI tự động kích hoạt mỗi khi có lệnh push hoặc pull_request gửi vào nhánh main.

3.1 Thiết lập Branch Protection trên GitHub:
Vào Settings → Branches → Add branch protection rule cho nhánh main.
Bật 'Require a pull request before merging' và 'Require status checks to pass before merging', chọn bắt buộc các job:

Code Quality & Linting, Unit & Integration Tests (CPU), Docker Build Validation.

3.2 Kịch bản kiểm chứng Pull Request lỗi (Trạng thái ĐỎ):
Khi một lập trình viên tạo PR chứa lỗi unit test (ví dụ gửi mốc thời gian không đủ 72 bước), pipeline CI sẽ lập tức báo đỏ FAILED và GitHub tự động khóa nút Merge Pull Request, ngăn chặn lỗi lọt vào nhánh chính.

3.3 Kịch bản khắc phục và nghiệm thu (Trạng thái XANH):
Sau khi lập trình viên sửa lỗi và đẩy commit bổ sung, GitHub Actions tự động kiểm thử lại toàn bộ 49 test cases. Khi toàn bộ jobs hiển thị dấu tích xanh PASSED, nút Merge Pull Request sẽ mở khóa cho phép tích hợp.

4. Quy Trình CD Tự Động: Release Lên Docker Hub

Pipeline CD được định nghĩa tại .github/workflows/release.yml, chỉ chạy khi có Git Tag hợp lệ dạng v*.*.*.

Lệnh tạo và đẩy Git Tag Release:

```
git checkout main
git pull origin main
git tag -a v1.0.0 -m "Release v1.0.0: DINOv3 Forecaster & Serving Container"
git push origin v1.0.0
```

Các bước tự động của CD Pipeline:
1. Pre-Release Quality Gate: Chạy kiểm định toàn bộ 49 bài test trước khi cho phép build.
2. Metadata Extraction: Tự động sinh các tag v1.0.0, 1.0, và latest.
3. Build & Push: Đăng nhập bằng token, build container và đẩy lên Docker Hub.
4. GitHub Summary: Ghi nhận mã băm bất biến (SHA-256 Digest) vào bản tóm tắt phát hành.

5. Hướng Dẫn Kéo & Chạy Image Trên Máy Trạm Sạch

Người đánh giá (Reviewer) có thể chạy thử nghiệm trên máy sạch độc lập mà không cần clone source code:

5.1 Kéo Image từ Docker Hub:

```
docker pull nguyenthuanhoa/hanoi-temperature:v1.0.0
```

5.2 Khởi chạy Container với Volume Mount Runtime Checkpoint:

```
# Chạy container với mount checkpoints và artifacts chế độ chỉ đọc (:ro)
docker run -d --name hanoi-weather-api -p 8000:8000 \
-v "$(pwd)/checkpoints:/app/checkpoints:ro" \
-v "$(pwd)/artifacts:/app/artifacts:ro" \
--restart unless-stopped \
nguyenthuanhoa/hanoi-temperature:v1.0.0
```

5.3 Kiểm tra xác thực quyền Non-Root:

docker exec hanoi-weather-api whoami
Kết quả mong đợi: appuser (uid=10001, gid=10001)

5.4 Kiểm tra thực tế bằng lệnh Curl:

```
# 1. Kiểm tra sức khỏe dịch vụ:
curl -X GET http://localhost:8000/health

# 2. Truy vấn siêu dữ liệu mô hình & 6 trạm khí tượng:
curl -X GET http://localhost:8000/metadata

# 3. Gửi payload dự báo nhiệt độ đồng thời 6 trạm sau 60 phút:
curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d @docs/api_example.json
```

6. Bảng Xử Lý Sự Cố & Rollback Khẩn Cấp

Hiện tượng / Lỗi	Nguyên nhân	Giải pháp xử lý dứt điểm
Lỗi 403 Forbidden khi push Docker	Token hết hạn hoặc sai username trong Secrets	Tạo lại Token trên Docker Hub với quyền Write, cập nhật GitHub Secrets.
HTTP 422: Timesteps != 72	Chuỗi thời gian lookback không đủ 6 giờ (72 mốc)	Gửi đúng 72 mốc thời gian liên tục với chu kỳ 5 phút.
Xung đột cổng 8000 trên host	Đã có dịch vụ khác đang chạy cổng 8000	Đổi port mapping trên máy host: -p 8080:8000.
Cần Rollback phiên bản khẩn cấp	Phiên bản mới gặp lỗi bất ngờ trên production	Khởi chạy lại image bằng SHA-256 Digest bất biến trước đó; tuyệt đối không xóa image cũ trên Docker Hub.